MINI PROJECT USE CASES

1.SQL: Digital Library Audit

Business Case: A local community college has a database of books and student borrows.

They are struggling to track Overdue books and want to know which categories of books are

most popular to decide what to buy next.

Problem Statement

Create a relational system to track book loans and generate a Penalty Report for books not

returned within 14 days.

Student Tasks:

1. Table Creation: Create Books, Students, and IssuedBooks (with IssueDate and

ReturnDate).

2. Overdue Logic: Write a query to find all students who havent returned a book where

the IssueDate was more than 14 days ago and ReturnDate is NULL.

3. Popularity Index: Use COUNT and GROUP BY on the Category column to show which

genre (e.g., Fiction, Science, History) is borrowed the most.

4. Data Cleanup: Write a DELETE or UPDATE statement to remove student records who

Haven

t borrowed a book in over 3 years (Inactive accounts).

Deliverable: A .sql file containing the DDL (table creation) and the analytical queries.

Solution

```
CREATE TABLE books (
   book_id INTEGER PRIMARY KEY,
   title TEXT,
   author TEXT,
   category TEXT
);

CREATE TABLE students (
   student_id INTEGER PRIMARY KEY,
   student_name TEXT,
   email TEXT,
   join_date DATE
```

```
);

CREATE TABLE issued_books (
    issue_id INTEGER PRIMARY KEY,
    book_id INTEGER,
    student_id INTEGER,
    issue_date DATE,
    return_date DATE,
    FOREIGN KEY (book_id) REFERENCES books(book_id),
    FOREIGN KEY (student_id) REFERENCES students(student_id)
);
INSERT INTO books VALUES (1, 'Java Basics', 'James', 'Programming');
INSERT INTO books VALUES (2, 'Python Guide', 'Guido', 'Programming');
INSERT INTO books VALUES (3, 'Data Science', 'Andrew', 'AI');

INSERT INTO students VALUES (101, 'Pooja', 'pooja@gmail.com', '2022-01-10');
INSERT INTO students VALUES (102, 'Ravi', 'ravi@gmail.com', '2021-05-15');

INSERT INTO issued_books VALUES (1, 1, 101, '2024-03-01', NULL);
INSERT INTO issued_books VALUES (2, 2, 102, '2022-02-01', '2022-02-10');

SELECT * FROM books;
SELECT * FROM students;
SELECT * FROM issued_books;

SELECT
    s.student_id,
    s.student_name,
    b.title,
    i.issue_date
FROM issued_books i
```

```
JOIN students s ON i.student_id = s.student_id
JOIN books b ON i.book_id = b.book_id
WHERE i.return_date IS NULL
AND i.issue_date < DATE('now', '-14 days');

SELECT
   b.category,
   COUNT(*) AS total_borrowed
FROM issued_books i
JOIN books b ON i.book_id = b.book_id
GROUP BY b.category
ORDER BY total_borrowed DESC;

ALTER TABLE students ADD status TEXT;

UPDATE students
SET status = 'Inactive'
WHERE student_id NOT IN (
   SELECT DISTINCT student_id
   FROM issued_books
   WHERE issue_date >= DATE('now', '-3 years')
);
```

2.After CREATE TABLE

```
SELECT name FROM sqlite_master WHERE type='table';
```

After INSERT DATA

SELECT * FROM books;

```
Detach file from library | ▷Run on active connection
1   SELECT * FROM books;
```

book_id	title	author
1	Java Basics	James
2	Python Guide	Guido
3	Data Science	Andrew

SELECT * FROM students;

student_id	student_name	email	join_date
101	Pooja	pooja@gmail.com	2022-01-10
102	Ravi	ravi@gmail.com	2021-05-15

SELECT * FROM issued_books;

issue_id	book_id	student_id	issue_date	return_date
1	1	101	2024-03-01	NULL
2	2	102	2022-02-01	2022-02-10

Overdue Books Query

```
SELECT
s.student_id,
s.student_name,
b.title,
i.issue_date
FROM issued_books i
```

```
JOIN students s ON i.student_id = s.student_id
JOIN books b ON i.book_id = b.book_id
WHERE i.return_date IS NULL
AND i.issue_date < DATE('now', '-14 days');
```

student_id	student_name	title	issue_date
101	Pooja	Java Basics	2024-03-01

Most Borrowed Category

```
SELECT
   b.category,
   COUNT(*) AS total_borrowed
FROM issued_books i
JOIN books b ON i.book_id = b.book_id
GROUP BY b.category
ORDER BY total_borrowed DESC;
```

category	total_borrowed
Programming	2

After ALTER TABLE

```
PRAGMA table_info(students);
```

cid	name	type	notnull	dflt_value	pk
0	student_id	INTEGER	0	NULL	1
1	student_name	TEXT	0	NULL	0
2	email	TEXT	0	NULL	0
3	join_date	DATE	0	NULL	0
4	status	TEXT	0	NULL	0

After UPDATE

SELECT student_id, student_name, status FROM students;

student_id	student_name	status
101	Pooja	NULL
102	Ravi	Inactive

2.Core Java: The Password Validator

Business Case: A cybersecurity firm needs a tool for their "Employee Portal" that forces

employees to create strong passwords. Standard "if-else" isn't enough; they need a modular

approach.

Problem Statement

Build a Password Strength Checker that validates a string against corporate security policies

and provides specific feedback on why a password failed.

Student Tasks:

1. The Policy: The password must be:

○ At least 8 characters long.

○ Contain at least one Uppercase letter.

○ Contain at least one Digit (0-9).

2. Looping Logic: Use a for loop to iterate through the string and

Character.isUpperCase() / Character.isDigit() to check requirements.

3. Feedback System: Instead of just saying "Invalid," the program should print specifically:

"Missing a digit" or "Too short."

4. Retry Mechanism: Use a while loop to keep asking the user for a password until they

enter a valid one.

Deliverable: A single PasswordValidator.java file that demonstrates string manipulation and

loop control.

SOLUTION

```
import java.util.Scanner;


public class PasswordValidator {


public static boolean isValidPassword(String password) {
    boolean hasUpper = false;
    boolean hasDigit = false;
    if (password.length() < 8) {
        System.out.println("Password is too short (minimum 8 characters required)");
        return false;
    }
    for (int i = 0; i < password.length(); i++) {
        char ch = password.charAt(i);


        if (Character.isUpperCase(ch)) {
            hasUpper = true;
        }
        if (Character.isDigit(ch)) {
```

```
            hasDigit = true;
        }
    }

    if (!hasUpper) {
        System.out.println("Missing an uppercase letter");
    }

    if (!hasDigit) {
        System.out.println("Missing a digit");
    }

    return hasUpper && hasDigit;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    String password;

    while (true) {
        System.out.print("Enter your password: ");
        password = sc.nextLine();

        if (isValidPassword(password)) {
            System.out.println("Password is strong and accepted!");
            break;
        } else {
            System.out.println(" Please try again.\n");
        }
    }
```

```
    sc.close();
}

}
```

```
PS D:\mini project - virtusa> java PasswordValidator.java
Enter your password: pooja123
Missing an uppercase letter
 Please try again.

Enter your password: Pooja
Password is too short (minimum 8 characters required)
 Please try again.

Enter your password: Poojamm
Password is too short (minimum 8 characters required)
 Please try again.

Enter your password: poojamala
Missing an uppercase letter
Missing a digit
 Please try again.

Enter your password: 12345678
Missing an uppercase letter
 Please try again.

Enter your password: Pooja123
Password is strong and accepted!
```

3.Core Python: The Fare Travel Optimizer

Business Case: A ride-sharing startup, "CityCab," needs a backend script to calculate fares.

The fare isn't flat; it changes based on the time of day (Peak Hours) and the type of vehicle requested.

Problem Statement

Write a script that calculates the final "Ride Estimate" based on distance, vehicle type, and a

"Surge Pricing" multiplier.

Student Tasks:

1. Dictionary Mapping: Store rates in a dictionary: {'Economy': 10, 'Premium': 18, 'SUV':

25} (rates per km).

2. Surge Logic: Ask the user for the "Hour of Day" (0-23). If the hour is between 17 and 20

(5 PM - 8 PM), apply a 1.5x Surge Multiplier to the total.

3. Function Definition: Create a function calculate_fare(km, type, hour) that returns the

final price.

4. Error Handling: If the user enters a vehicle type not in your dictionary, use a try-except

block or an if-in check to provide a "Service Not Available" message.

Deliverable: A .py script that takes user input and prints a formatted "Price Receipt."

SOLUTION :

```
rates = {
    "economy": 10,
    "premium": 18,
    "suv": 25
}


def calculate_fare(distance, vehicle_type, hour):
    if vehicle_type not in rates:
        return None


    rate_per_km = rates[vehicle_type]
    base_fare = distance * rate_per_km


    if 17 <= hour <= 20:
        surge_multiplier = 1.5
    else:
```

```
        surge_multiplier = 1

    return base_fare * surge_multiplier

print("Welcome to CityCab Fare Calculator")
print("------------------------------------")

try:
    distance = float(input("Enter distance (in km): "))
    vehicle_type = input("Enter vehicle type (Economy / Premium / SUV): ").strip().lower()
    hour = int(input("Enter hour of travel (0-23): "))

    fare = calculate_fare(distance, vehicle_type, hour)

    if fare is None:
        print("Service Not Available for this vehicle type")
    else:
        print("\n========= PRICE RECEIPT =========")
        print(f"Distance       : {distance} km")
        print(f"Vehicle Type    : {vehicle_type.capitalize()}")
        print(f"Travel Hour     : {hour}")

        if 17 <= hour <= 20:
            print("Surge Pricing   : Applied (1.5x)")
        else:
            print("Surge Pricing   : Not Applied")

        print(f"Total Fare      : ₹{fare:.2f}")
        print("================================")

except ValueError:
```

print("Invalid input! Please enter correct values.")

```
PS D:\mini project - virtusa> python farecalc.py
Enter vehicle type (Economy / Premium / SUV): premium
Enter hour of travel (0-23): 20

========= PRICE RECEIPT =========
Distance         : 10.0 km
Vehicle Type     : Premium
Travel Hour      : 20
Surge Pricing    : Applied (1.5x)
Total Fare       : ₹270.00
==================================
PS D:\mini project - virtusa> python farecalc.py
Welcome to CityCab Fare Calculator
-------------------------------------
Enter distance (in km): 10
Enter vehicle type (Economy / Premium / SUV): gold
Enter hour of travel (0-23): 21
Service Not Available for this vehicle type
PS D:\mini project - virtusa> python farecalc.py
Welcome to CityCab Fare Calculator
-------------------------------------
Enter distance (in km): 10
Enter vehicle type (Economy / Premium / SUV): economy
Enter hour of travel (0-23): 15

========= PRICE RECEIPT =========
Distance         : 10.0 km
Vehicle Type     : Economy
Travel Hour      : 15
Surge Pricing    : Not Applied
Total Fare       : ₹100.00
==================================
```